Kahawa Roots

Traceability Platform

Tracking Coffee from Kenya's Farms to the UK

Shree Hari Anbazhagan

shreeharianbazhagan@gmail.com
07721046613

Pre-Interview Task — Developer Intern — April 2026

Proof: I've Built This Before

My live full-stack project on the **same GCP infrastructure** I'm proposing:

- DeltaEd — Live on GCP Cloud Run
- GCP Cloud Run, `europe-west2` (London)
- Dockerized, full stack
- React + Python + BigQuery

happy to walk through the architecture if interested.

How I'd Build This

How Data is Stored & Recorded

- every shipment gets a unique ID when first recorded
- raw data (form/CSV) → **Cloud Storage** as immutable backup
- **Cloud Function** auto-triggers, cleans & validates
- clean data → **Cloud SQL (PostgreSQL)** — 3 tables: `farmers`, `shipments`, `shipment_stages`
- each stage (farm → processing → export → UK) is a row with timestamps and GPS

QR Code → Webpage

- QR generated per shipment, encodes URL like `/trace/SH-2026-001`

Non-Technical Data Entry

Option 1: Web Form

- simple form on admin dashboard
- one shipment at a time
- no coding, no terminal, no SQL

Option 2: CSV/Excel Upload

- drag and drop a spreadsheet
- bulk import for cooperative batch data
- auto-validates every row

both feed into same REST API → same validation → same DB.

Technology Choices + Bonus

Tech Stack & Why

Component	Choice & Why
Cloud	**GCP** — London region, **$200K credits**, GDPR, scales to zero
Backend	**Python + REST API** on Cloud Run
Frontend	**Next.js (React)** — SSR, fast QR loads
Database	**Cloud SQL (PostgreSQL)** — sub-10ms reads
Storage	**Cloud Storage** — immutable raw backup
Processing	**Cloud Functions** — event-driven
IaC	**Terraform** — repeatable infra
CI/CD	**GitHub Actions** — auto deploy
Maps	**Google Maps API** — native to GCP

Medallion Architecture

Bronze Layer
Raw CSV/JSON in Cloud Storage

↓

Silver Layer
Clean data in PostgreSQL

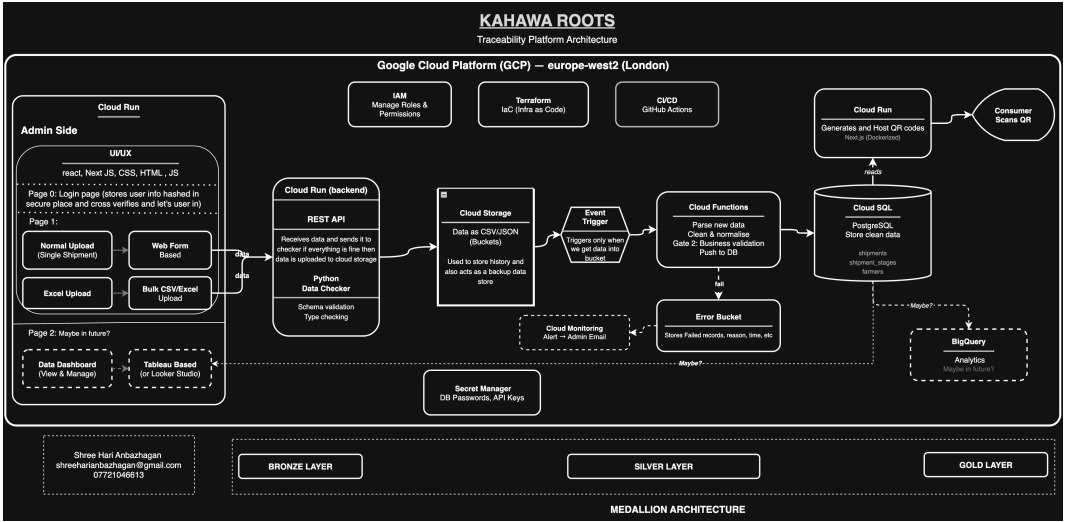
↓

Gold Layer
QR Pages + BigQuery (future)

Bonus: Database Schema

```
CREATE TABLE farmers (
  id UUID PK, name VARCHAR, region VARCHAR,
  farm_size DECIMAL, lat/lon DECIMAL(9,6),
  cooperative VARCHAR );
CREATE TABLE shipments (
  id UUID PK, batch_id VARCHAR UNIQUE,
  farmer_id UUID FK→farmers, grade VARCHAR,
  weight_kg DECIMAL, status VARCHAR,
  created_at TIMESTAMP DEFAULT NOW() );
CREATE TABLE shipment_stages (
  id UUID PK, shipment_id UUID FK→shipments,
  stage VARCHAR, location VARCHAR, notes TEXT );
```

System Architecture



🔗 Click here for interactive architecture diagram

What Does All This Actually Mean?

Architecture Concepts

- **Microservices** — instead of one big app, we split into 3 small independent ones (admin, API, QR page). if one breaks, the others keep running.
- **Cloud Functions** — code that runs *only* when new data arrives. no server to manage, no cost when idle.
- **REST API** — the "messenger" between the admin form and database. checks, cleans, and passes data along.
- **PostgreSQL** — a database like a smart spreadsheet. stores shipments in organised tables with relationships.
- **Medallion Architecture** — data flows through 3 layers: **Bronze** (raw backup) → **Silver** (cleaned) → **Gold** (customer-facing). no bad data reaches the QR page.

Operations & Security

- **IAM** — controls *who* can do *what*. admins add shipments, public can only view QR pages.
- **Terraform (IaC)** — infra defined in code, not clicked manually. one command rebuilds the whole system.
- **CI/CD** — every code push is auto-tested and deployed. no manual uploads, no "works on my laptop" issues.
- **Secret Manager** — stores passwords & API keys securely. no accidental credential leaks.

Future Plans

- **BigQuery** — analytics when we have enough data. "fastest shipping regions?" "transit time trends?"
- **Dashboard** — Tableau/Looker Studio for founders to see charts without touching the database.